K / V

KHANIVERSITY

MASTER DEVELOPMENT BLUEPRINT

AI-Native University Architecture | Computer Science Academy MVP

LEARN. BUILD. INNOVATE. MASTER.

One intelligent university. Multiple specialist schools. One focused first release.

Document ID	KH-001
Version	1.0
Publication	October 2026
Status	Strategic development blueprint | Proposed design

KHANIVERSITY / COMPUTER SCIENCE ACADEMY

A project architecture, academic plan, and implementation roadmap prepared for initial development.

DOCUMENT OVERVIEW

Contents

01 Executive vision and scope
02 University and school structure
03 Complete system architecture
04 Technology stack
05 Academic model and CS101 curriculum
06 AI Professor
07 Data architecture
08 Platform interfaces
09 Roles and access control
10 Security, privacy and academic status
11 MVP roadmap and success metrics
12 Repository and engineering foundations
13 Long-term expansion
14 Release recommendation
A Standards and reference links

DOCUMENT CONTROL

Field	Specification
Owner	KHANIVERSITY product and engineering initiative
Primary deliverable	Hybrid institutional blueprint and Computer Science Academy MVP
Scope	Strategic, academic, software, security and operational architecture
Interpretation	Recommendations and planning targets; not completed implementation or accreditation

01 Executive vision and scope

KHANIVERSITY is proposed as an AI-native digital learning ecosystem that progresses students from foundational knowledge to demonstrable professional competency through structured instruction, intelligent tutoring, practical laboratories, competency evaluation and real-world projects.

The hybrid development approach establishes a scalable university-wide architecture while limiting the first operating release to the School of Computer Science. The platform will be structured to support future schools without building unnecessary services, courses or administrative infrastructure before product-market and pedagogical validation.

Strategic outcome

North-star outcome. A learner can register, enroll in Computer Science Foundations, complete the full CS101 pathway, execute and submit Python code, receive contextual AI tutoring, pass competency assessments, finish a

capstone project and view a trustworthy achievement record.

Decisions at a glance

Dimension	Initial decision
Audience	Adult beginner-to-intermediate computer science learners
Initial channel	Responsive web application; progressive-web-app readiness
Learning methodology	Self-paced, mastery-oriented and AI-assisted
First school	School of Computer Science
First program	Computer Science Foundations
First coding language	Python
Differentiator	Grounded AI Professor + executable CodeLab + validated assessments
Pilot commercial model	Free controlled pilot; monetization after validation
Engineering approach	Modular monolith with well-defined domain boundaries
Credentialing	Internal completion achievements; no claim of accredited degrees

02 University and school structure

The KHANIVERSITY platform shall provide common identity, academic catalog, assessment, tutoring, access-control and operational systems. Schools are academic catalog entities governed by shared platform infrastructure, not separate software deployments.

School	Proposed status	Curricular domain
Computer Science	MVP - first school	Programming, computation, algorithms, systems
Software Engineering	Future	Web/mobile, APIs, architecture, testing, DevOps
Artificial Intelligence	Future	Machine learning, deep learning, LLMs, agentic AI
Cybersecurity	Future	Networking, defensive security, forensics, cryptography
Cloud & Infrastructure	Future	Linux, databases, containers and cloud services
Emerging Technologies	Future	Robotics, IoT, embedded and next-generation computing

Institutional hierarchy

KHANIVERSITY
University-wide governance, shared identity and technology infrastructure

▼

Specialist schools
Six catalog-level schools planned; one active for MVP

▼

Programs and courses
Structured program outcomes, prerequisites and course sequences

▼

Modules, lessons and learning objectives
Versioned educational content connected to measurable competencies

▼

Exercises, assessments and projects
Observable evidence of student understanding and skill

Each school should be able to use the common platform without duplicating authentication, learning analytics, AI usage tracking, or academic records.

03 Complete system architecture

The initial platform is one deployable Next.js application containing independently organized modules. This reduces infrastructure cost and operational friction while preserving separation of academic, learning, assessment, AI, coding and administrative responsibilities.

Logical architecture

Presentation layer
Public website | Student Portal | Learning Studio | CodeLab | Admin Console

▼

Application/API layer
Next.js server modules | authenticated routes | validation | authorization

▼

Domain modules
Academic | Learning | Assessment | AI Tutor | CodeLab | Operations

▼

Platform services
PostgreSQL | AI provider APIs | file storage | Pyodide runtime | observability

Responsibilities and boundaries

Module	Responsibility	Core design boundary
Academic	Schools, programs, curriculum and versioning	Publishes approved content

Module	Responsibility	Core design boundary
Learning	Enrollment, progress, prerequisite and mastery state	Owns learner progression
Assessment	Quizzes, rubrics, submissions and grading	Owns authoritative results
AI Tutor	Grounded guidance, hints and conversational records	Cannot directly overwrite grades
CodeLab	Editor, execution, test results and exercise drafts	Runs untrusted code in isolation
Operations	Permissions, audit trails, operational analytics	Administrative changes are logged

Explicit architectural exclusions from the MVP

- Do not split the six domains into separately deployed microservices.
- Do not introduce Kubernetes, a distributed event bus, or custom identity management without demonstrated need.
- Do not provision full container-based coding infrastructure until browser-based Python proves insufficient.
- Do not implement all future school modules before the first Computer Science pathway works end to end.

04 Technology stack

Layer	Selected technology	Why it is selected
Web application	Next.js, React, TypeScript	Routing, rendering, type-safe application development
UI / design system	Tailwind CSS, shadcn/ui	Reusable accessible interface components
Backend	Next.js server modules	Simple deployable boundary for the MVP
Authentication	Supabase Auth	Managed identity and sessions
Database	Supabase PostgreSQL	Relational records, SQL integrity and RLS
Authorization	PostgreSQL RLS + server guards	Data ownership and least-privilege access
AI orchestration	Vercel AI SDK + provider API	Model abstraction, streaming and tool orchestration
Knowledge retrieval	PostgreSQL + pgvector	Ground AI lessons in approved content
Python runtime	Pyodide in a Web Worker	Basic Python execution without per-user containers
Future JS coding	Sandpack	Browser-based JavaScript/React exercises
Course files	Supabase Storage	Managed educational assets
Hosting	Vercel	Web delivery and deployment previews
CI/CD	GitHub Actions	Automated testing and build gates
Observability	Structured logs and error monitoring	Failures, errors and usage signals

Code execution decision

MVP execution model. Use Pyodide inside a dedicated Web Worker for foundational Python exercises. Keep main-thread UI responsive, restrict available integrations, and treat user code as untrusted. For later projects requiring operating-system processes, server frameworks or extensive packages, design a separately isolated container service with strict CPU, memory, network and time limits.

05 Academic model and CS101 curriculum

Learning content must be organized as School → Program → Course → Module → Lesson → Learning Objective. Each learning objective maps to instructional material and one or more pieces of observable assessment evidence.

The long-term curricular reference should include ACM/IEEE-CS/AAAI CS2023 guidance as a quality benchmark; adherence to curriculum guidance alone does not imply institutional accreditation.

First program

Attribute	Program definition
School	Computer Science
Program	Computer Science Foundations
Pilot course	CS101 - Introduction to Computer Science & Python
Delivery	Self-paced, mobile-responsive, practical
Prerequisites	No programming experience assumed
Competency evidence	Quizzes, coding tests and capstone rubric
Completion rule	Proposed: ≥80% quiz threshold, required code tests passed and capstone rubric passed

CS101 - 10-module pilot syllabus

Module	Topic	Expected demonstrated ability
01	Computer systems and computational thinking	Describe computational processes and represent a solution as steps
02	Programming fundamentals and Python syntax	Write and execute a simple Python script
03	Variables, data types and input/output	Process and display typed input correctly
04	Conditions, loops and control flow	Solve a branching and iteration problem
05	Functions, scope and decomposition	Break a program into reusable functions
06	Lists, tuples, dictionaries and sets	Choose and use appropriate data structures
07	Files, exceptions and debugging	Identify faults and recover from common errors

Module	Topic	Expected demonstrated ability
08	Algorithms, complexity and problem solving	Explain basic efficiency trade-offs
09	Testing, Git fundamentals and code quality	Run checks, track revisions and improve readability
10	Capstone and practical examination	Design, build, test and explain a working program

Standard lesson composition

- Learning outcome, prerequisites and learner-facing explanation.
- Worked examples with progressive difficulty and annotated execution.
- Guided exercises with increasing independence and scaffolded hints.
- Formative AI Tutor interactions grounded in the active lesson.
- Automated tests, assessment rubric and a remediation route when mastery is not yet met.

Capstone project: Personal Study Planner

The student builds a Python program that records study tasks, assigns priorities, persists progress and generates study summaries. Assessment criteria should cover feature completeness, valid input handling, data organization, test coverage, code readability and an oral or written explanation of key design decisions.

06 AI Professor

The AI Professor is a pedagogical engine rather than a generic chatbot. It uses approved lesson knowledge and constrained student context to diagnose misconceptions, teach concepts, guide practice and explain assessment feedback.

Instructional loop



Permitted tutor context

Input context	Purpose / treatment
Current published lesson	Primary grounding for explanations
Learning objectives and prerequisites	Keep content aligned with curriculum
Relevant assessment feedback	Target misconceptions and next actions
Current code and test output	Explain mistakes without assuming code was correct
Historical progress summary	Adjust difficulty and pacing
Learner preferences	Change tone or method, not academic standards

AI governance and integrity

- Formal scores originate from deterministic tests or instructor-approved rubrics, not unreviewed chat responses.
- The tutor must not expose hidden tests, secret answers or full graded solutions while an assessment is active.
- Retrieval must prioritize approved and versioned course content, with clear handling for uncertainty.
- AI messages require rate limiting, privacy controls, traceable model metadata and structured quality evaluation.
- A reviewer should be able to resolve disputed or subjective grading decisions.

07 Data architecture

Use a normalized relational schema with explicit foreign keys, authorization policies and auditability. Create only the tables needed by the initial deliverable; retain the remainder as a planned domain model.

Domain	Proposed primary tables
Identity	profiles, roles, user_roles
Academic catalog	schools, programs, courses, modules, lessons
Curriculum	learning_outcomes, lesson_outcomes, content_versions
Enrollment	enrollments, lesson_progress, mastery_records
Assessment	assessments, questions, submissions, test_cases
CodeLab	coding_exercises, code_submissions
AI tutoring	tutor_sessions, tutor_messages, ai_usage
Credentials	credentials, credential_evidence
Operations	audit_logs, feature_flags

Schema rules

- Keep curriculum content versioned; a later lesson edit must not silently redefine a student’s historical evaluation.
- Protect user and grade records through both server-side authorization and PostgreSQL Row Level Security.
- Keep system-owned test cases and assessment solutions inaccessible to ordinary student queries.

- Record instructor overrides and credential issuance as auditable events.
- Retain AI conversation data only for a documented purpose and defined period.

08 Platform interfaces

Product surface	MVP experiences
Public site	Institution overview, computer science school, course description, admissions and sign-in
Student Portal	Dashboard, enrollment, progress tracking and next recommended lesson
Learning Studio	Teaching materials, interactive questions, tutor interface and navigation
CodeLab	Editor, Python execution, console, automated checks, saved submissions
Academic Administration	Course drafting/publishing, user permissions, assignment review and analytics

Brand direction

Use a disciplined academic visual identity: deep navy foundations, restrained gold accents, light neutral surfaces, high-contrast typography, and uncluttered layout. The platform should communicate institutional credibility and engineering seriousness rather than imitate a generic conversational AI dashboard.

Accessibility expectations

Design keyboard-navigable workflows, readable code and feedback states, semantic structure, adequate contrast, clear errors and responsive layouts. CodeLab, quizzes and learner progress components must be usable without requiring pointer-only interaction.

09 Roles and access control

Role	Core permissions
Student	Enroll, learn, practice, submit and inspect personal results
Instructor	Author assigned course content and review designated student submissions
Academic reviewer	Approve course and assessment updates
Administrator	Manage catalog operations, staff permissions, audits and access policies

An account may hold multiple explicit roles; effective access is bounded by school, course and record ownership. Privileged operations require server checks in addition to UI restrictions. Service secrets must never be shipped to the browser.

10 Security, privacy and academic status

Control domain	Mandatory approach
Authentication	Managed sessions, verified recovery and MFA for privileged accounts
Authorization	Least privilege, RLS, tenant/record scoping and cross-account tests
Code execution	Worker isolation for browser Python; separate locked-down containers when server execution is introduced
AI security	Prompt-injection resistance, retrieval trust boundaries, strict model tool permissions and output validation
Privacy	Encryption in transit, published notices, retention, export/deletion procedures and lawful processing
Assessment integrity	Protected hidden tests, server-owned grading, logs for overrides
Resilience	Backups, error monitoring, rate limits, tested recovery procedures and release rollback plans

Kenyan data protection and regulatory considerations

For a Kenya-based public operation, assess the Office of the Data Protection Commissioner's requirements, including applicable controller/processor registration, notices, cross-border processing arrangements and handling of learner rights. Obtain jurisdiction-specific legal advice before broad deployment where necessary.

KHANIVERSITY must initially be represented as an online learning academy or platform, not an accredited degree-awarding university. Offering regulated university awards or claiming accreditation requires the appropriate approvals, including engagement with Kenya's Commission for University Education.

Release gate. No critical security findings, no confirmed unauthorized cross-account access, and no false accreditation claims are acceptable in a public release. Privacy controls and a tested incident response path must be ready before real learners are onboarded.

11 MVP roadmap and success metrics

Sprint	Work package	Deliverable / acceptance signal
01	Foundation and design system	Working app shell, database migrations, CI, auth scaffolding, visual tokens
02	Identity and academic catalog	Student registration, school/program/course views and enrollment
03	Learning engine	Versioned CS101 content, lesson navigation, persisted progression
04	CodeLab and assessments	Pyodide execution, quizzes, submissions and deterministic tests
05	AI Professor	Grounded context-aware tutor, hints, feedback controls and evaluation
06	Pilot and release hardening	Complete CS101, security checks, accessibility review and learner pilot

End-to-end acceptance criterion

A new student, without administrator intervention, can create an account, enroll in CS101, complete lessons, execute Python, submit tests, receive context-aware AI guidance, complete the capstone and inspect an accurate personal competency record.

Pilot performance hypotheses

Metric	Proposed target	Interpretation
Pilot size	50 learners	Controlled initial cohort
First-lesson activation	70%	Share of enrolled learners starting lesson one
CS101 completion	40%	Initial hypothesis subject to testing
Critical security defects	0	Mandatory public release condition

These are provisional planning targets; they are neither measured performance nor guaranteed results. Define analytics events and denominators before interpreting the pilot.

Release-quality checks

- Build, lint, type-check, unit tests and integration tests succeed.
- End-to-end paths for registration, enrollment, completion and submission pass.
- Student-to-student data isolation and role boundaries are verified.
- Browser coding is tested for runaway loops, cancellation and resource limits.
- AI responses are reviewed for instructional accuracy, leakage of solutions and unsafe tool behavior.
- Curriculum content and scoring rules receive instructional review before release.

12 Repository and engineering foundations

Use a dedicated repository named khaniversity-platform with a domain-organized modular monolith. Put course content and source-controlled database migrations in the same repository initially to minimize coordination complexity.

```
khaniversity-platform/
├── src/
│   ├── app/
│   │   ├── (public)/
│   │   ├── (auth)/
│   │   ├── (student)/
│   │   ├── (staff)/
│   │   └── api/
│   ├── components/
│   │   ├── ui/
│   │   ├── learning/
│   │   ├── codelab/
│   │   └── professor/
│   ├── modules/
│   │   ├── academics/
│   │   ├── enrollments/
│   │   ├── assessments/
│   │   ├── tutoring/
│   │   └── analytics/
│   ├── lib/
│   │   ├── auth/
│   │   ├── database/
│   │   ├── ai/
│   │   └── validation/
│   └── workers/python.worker.ts
├── supabase/migrations/
├── supabase/tests/
├── curriculum/cs101/
├── tests/{unit,integration,e2e}/
├── docs/{architecture,product}/
├── .github/workflows/
└── AGENTS.md
```

AI-assisted engineering policy

The root AGENTS.md should document code conventions, service boundaries, mandatory tests, environment requirements, secrets handling, instructions for code-writing agents, and explicit prohibitions against bypassing authorization or disabling checks to make builds pass.

13 Long-term expansion

Future capability	Expansion strategy
Multiple schools	Activate new curricula using the existing school/program abstractions
Advanced laboratories	Provide container-based cloud labs and safe security simulations
Instructor and cohort operations	Extend assignment, class, review and moderation workflows
Career portfolio	Expose approved capstone projects, validated skills and portfolios
Digital credentials	Introduce standards-oriented, verifiable achievement records
Mobile/offline	Add packaged lessons, download support and native experiences where justified
Commercial offerings	Explore subscriptions, cohorts and organization partnerships after validation

For portability and verifier compatibility, evaluate the 1EdTech Open Badges 3.0 standard when implementing digital credentials. A platform certificate is not itself an accredited academic qualification.

14 Release recommendation

First engineering deliverable: KHANIVERSITY Foundation Release 0.1. Build the application shell, branded homepage, secure student dashboard, operational catalog and one navigable CS101 lesson first. Use that vertical slice to validate data flow, academic modeling, visual identity, and developer experience before investing in broader labs or AI orchestration.

The governing execution principle is to engineer for a multi-school university while shipping the smallest complete student learning experience that proves the product and educational model work.

Immediate next decisions

- Confirm the initial brand assets and component design language.
- Establish the GitHub repository, environments and project governance rules.
- Approve the CS101 learning outcomes and assessment blueprint.
- Implement the Foundation Release 0.1 vertical slice.
- Identify pilot instructors/reviewers and define learner consent and evaluation procedures.

A Standards and reference links

Selected official references to validate technical and educational design choices. Product capabilities, laws and documentation may change after publication.

- Next.js App Router | https://nextjs.org/docs/app
- Supabase Authentication | https://supabase.com/docs/guides/auth
- Supabase Row Level Security | https://supabase.com/docs/guides/database/postgres/row-level-security
- Supabase pgvector | https://supabase.com/docs/guides/database/extensions/pgvector
- Vercel AI SDK | https://ai-sdk.dev/docs/introduction
- Pyodide documentation | https://pyodide.org/en/stable/
- Sandpack documentation | https://sandpack.codesandbox.io/docs
- ACM CS2023 Curricula | https://csed.acm.org/
- OWASP Top 10 for LLM Applications | https://owasp.org/www-project-top-10-for-large-language-model-applications/
- Kenya Office of the Data Protection Commissioner | https://www.odpc.go.ke/
- Kenya Commission for University Education | https://www.cue.or.ke/
- 1EdTech Open Badges | https://www.1edtech.org/standards/open-badges